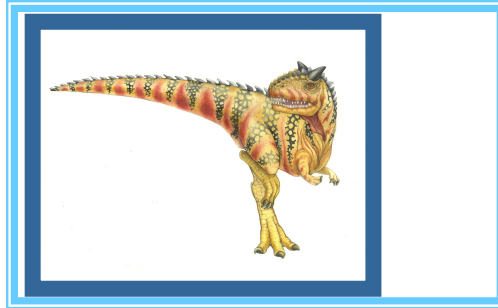


Chapter 6: CPU Scheduling





Chapter 6: CPU Scheduling

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multiple-Processor Scheduling
- Real-Time CPU Scheduling
- Operating Systems Examples
- Algorithm Evaluation





Objectives

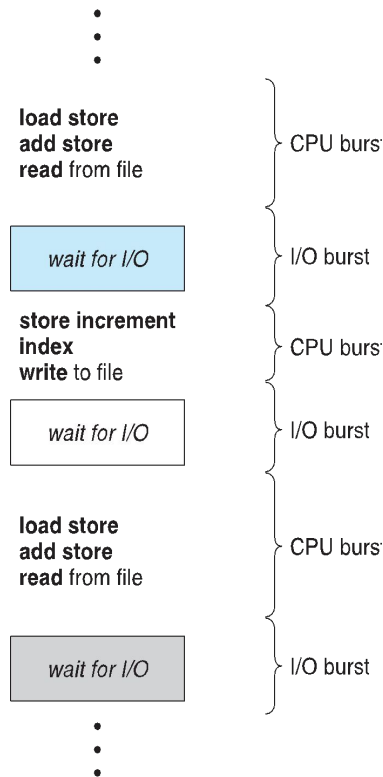
- To introduce CPU scheduling, which is the basis for multiprogrammed operating systems
- To describe various CPU-scheduling algorithms
- To discuss evaluation criteria for selecting a CPU-scheduling algorithm for a particular system
- To examine the scheduling algorithms of several operating systems





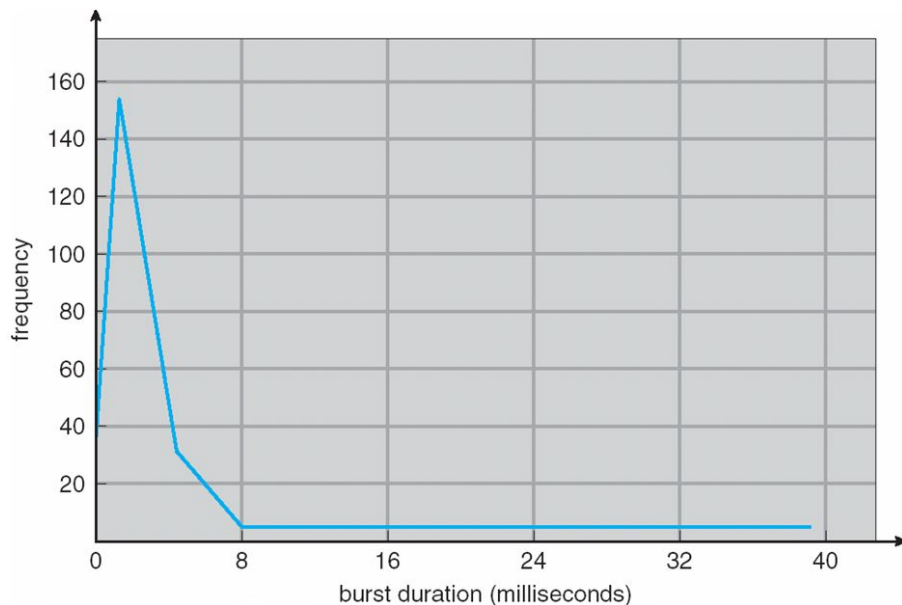
Basic Concepts

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- **CPU burst** followed by **I/O burst**
- CPU burst distribution is of main concern





Histogram of CPU-burst Times





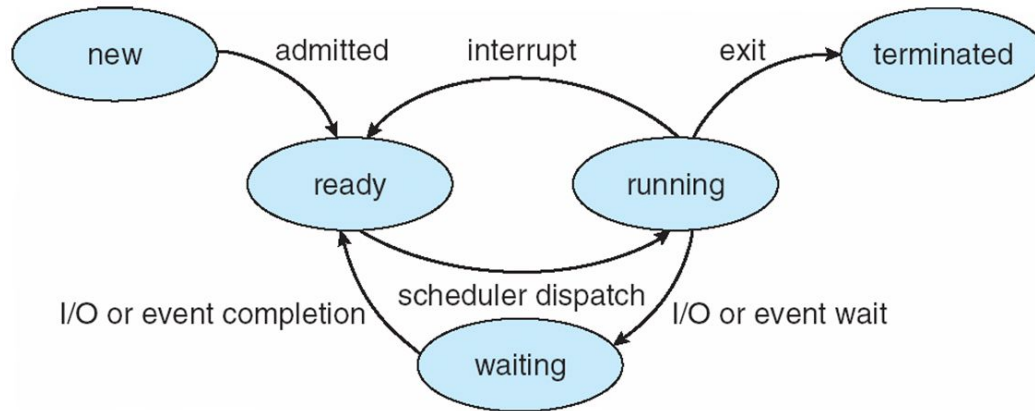
CPU Scheduler

- **Short-term scheduler** selects from among the processes in ready queue, and allocates the CPU to one of them
 - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state
 2. Switches from running to ready state
 3. Switches from waiting to ready
 4. Terminates
- Scheduling under 1 and 4 is **nonpreemptive (no interrupt)**
- All other scheduling is **preemptive (interrupt occurs)**





Diagram of Process State





Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - switching context
 - switching to user mode
 - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running





Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – Number of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process.
Completion Time - Arrival Time
- **Waiting time** – amount of time a process has been waiting in the ready queue. **Turnaround Time - Burst Time**
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)
Start of Process Execution - Arrival of Process





Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time





First- Come, First-Served (FCFS) Scheduling

Process ID	Burst Time	Arrival Time
P1	24	0
P2	3	0
P3	3	0

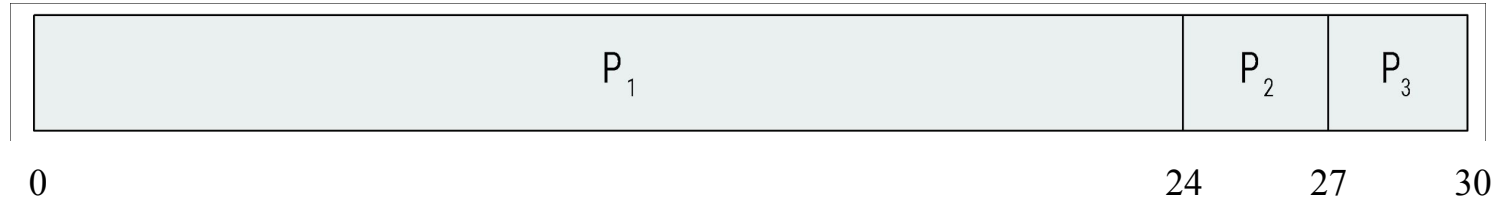
The order of arrival is P_1, P_2, P_3 . Calculate Average Waiting Time, Turnaround Time and Average Response Time.

Note: if arrival time same and order not mentioned, consider PID as priority (P_1 will go first among P_1, P_2, P_3 followed by P_2 and P_3)





First- Come, First-Served (FCFS) Scheduling



	Turnaround Times	Waiting Times	Response Times
P_1	24-0	24-24	0-0
P_2	27-0	27-3	24-0
P_3	30-0	30-3	27-0
Average	27	17	17





First- Come, First-Served (FCFS) Scheduling

The order of arrival is P_2 , P_3 , P_1 . Calculate Average Waiting Time, Turnaround Time and Average Response Time.



	Turnaround Times	Waiting Times	Response Times
P_1	30-0	30-24	6-0
P_2	3-0	3-3	0-0
P_3	6-0	6-3	3-0
Average	13	3	3





FCFS Scheduling (Cont.)

- Waiting Time, Turnaround Time and Response Time much better than previous case
- Bad for time sharing systems: convoy effect and high wait time.
- **Convoy effect** - short process behind long process
 - Consider one CPU-bound and many I/O-bound processes





FCFS Scheduling Problem

Process ID	Burst Time	Arrival Time
P1	5	4
P2	4	6
P3	3	0
P4	2	6
P5	4	5

Draw a Gantt chart of the processes. Calculate Average Waiting Time, Turnaround Time and Average Response Time.





Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst
 - Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives minimum average waiting time for a given set of processes
- Problem:
 - The difficulty is knowing the length of the next CPU request
- Solution:
 - So use a probabilistic estimation
 - Could ask the user

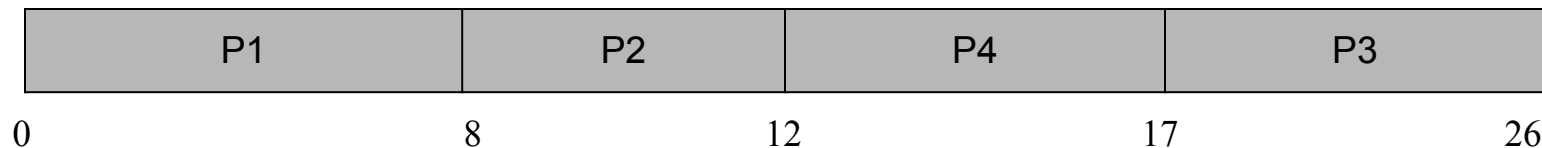




Example of SJF

Process ID	Arrival	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

- SJF scheduling chart



- Average waiting time = $(0 + 7 + 15 + 9) / 4 = 7.75$

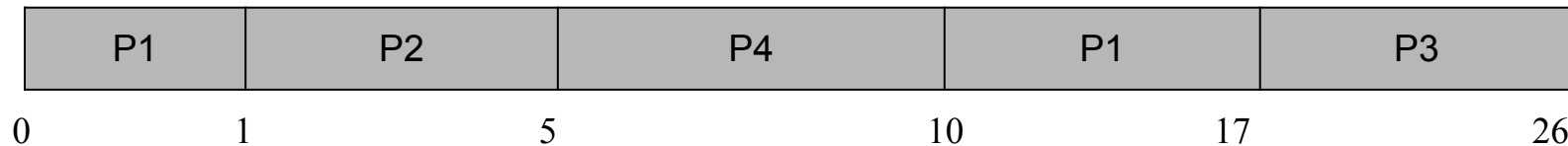




Example of Shortest-remaining-time-first

Process ID	Arrival	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

- *Preemptive* SJF Gantt Chart



- Average waiting time = $(9+0+15+2)/4 = 26/4 = 6.5$





Shortest-remaining-time-first Scheduling Problem

Process ID	Arrival Time	Burst Time
P1	0	10
P2	3	6
P3	7	9
P4	8	5

Draw a Gantt chart of the processes. Calculate Average Waiting Time, Turnaround Time and Average Response Time.





Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority)
 - Preemptive
 - Nonpreemptive
- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Problem $\equiv$ **Starvation** – low priority processes may never execute
- Solution $\equiv$ **Aging** – as time progresses increase the priority of the process

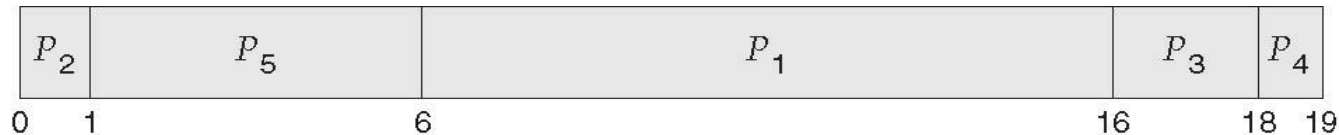




Example of Priority Scheduling

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Priority scheduling Gantt Chart





Priority Scheduling Problem

Process ID	Burst Time	Arrival Time	Priority
P1	11	0	2
P2	28	5	0
P3	2	12	3
P4	10	2	1
P5	16	9	4

Draw a Gantt chart of the processes. Calculate Average Waiting Time, Turnaround Time and Average Response Time.





Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum q**), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
- Timer interrupts every quantum to schedule next process
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high

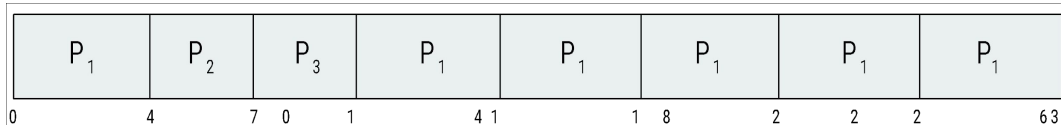




Example of RR with Time Quantum = 4

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- The Gantt chart is:

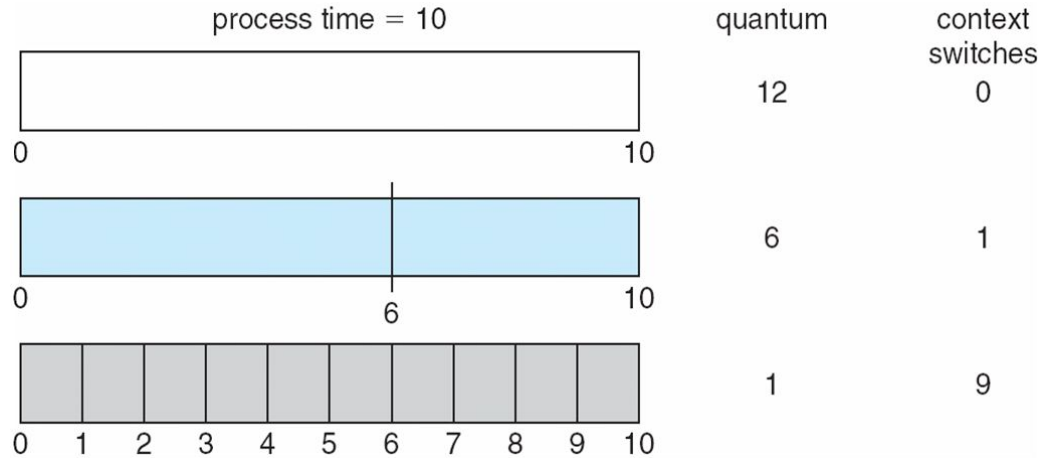


- Typically, higher average turnaround than SJF, but better *response*
- q should be large compared to context switch time





Time Quantum and Context Switch Time





Round Robin Problem (Quantum Time = 4)

Process ID	Burst Time	Arrival Time	Priority
P1	6	0	2
P2	10	5	0
P3	2	10	3
P4	12	2	1

Draw a Gantt chart of the processes. Calculate Average Waiting Time, Turnaround Time and Average Response Time.





Multilevel Queue

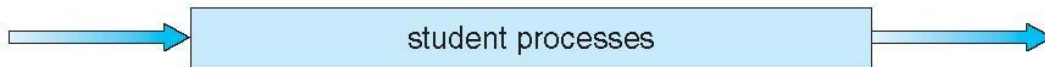
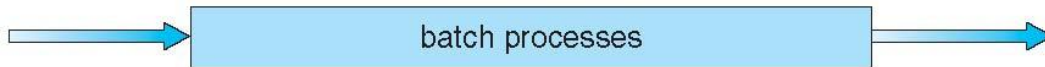
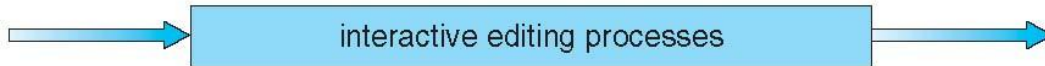
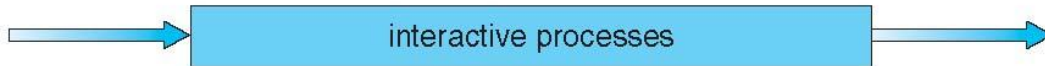
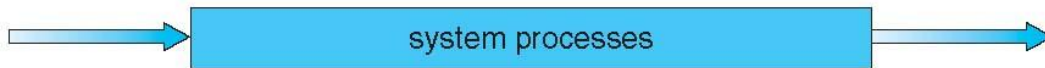
- Ready queue is partitioned into separate queues, eg:
 - **foreground** (interactive)
 - **background** (batch)
- Process permanently in a given queue
- Each queue has its own scheduling algorithm:
 - foreground – RR
 - background – FCFS
- Scheduling must be done between the queues:
 - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR
 - 20% to background in FCFS





Multilevel Queue Scheduling

highest priority



lowest priority





Multilevel Feedback Queue

- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service





Example of Multilevel Feedback Queue

- Three queues:
 - Q_0 – RR with time quantum 8 milliseconds
 - Q_1 – RR time quantum 16 milliseconds
 - Q_2 – FCFS
- Scheduling
 - A new job enters queue Q_0 which is served FCFS
 - When it gains CPU, job receives 8 milliseconds
 - If it does not finish in 8 milliseconds, job is moved to queue Q_1
 - At Q_1 job is again served FCFS and receives 16 additional milliseconds
 - If it still does not complete, it is preempted and moved to queue Q_2

